

RoadSOS

Technical Documentation

Architecture, Algorithms, and Implementation Reference

`github.com/Arthrevs/RoadSOS`

Branch: `main` Commit: `f94dc2a`

May 2026

Scope. This document is the engineering reference for RoadSOS. It covers the request lifecycle, the OpenStreetMap Overpass query layer, the Google Places fallback strategy, the four-tier offline cache hierarchy, the AI triage model, internationalization across 48 languages, the real-time map renderer, crash detection signals, and the deployment topology. Every number cited is grounded in source. Every external dependency is named. No marketing.

Contents

1	System Overview	4
1.1	Top-Level Topology	4
1.2	Tech Stack at a Glance	4
2	Backend: The <code>/search</code> Orchestrator	5
2.1	Phase 1 — Parallel Geocode and Overpass	5
2.1.1	The Overpass Query Layer	5
2.1.2	Multi-Mirror Retry Strategy	5
2.1.3	Element Parsing and Distance	6
2.1.4	Proximity Deduplication	6
2.2	Phase 2 — Conditional Google Places Fallback	6
2.2.1	Multi-Key Rotation	6
2.3	Phase 3 — Merge and Deduplicate	7
2.4	Phase 4 — Phone Enrichment	7
2.5	Response Shape	7
3	Backend: Caching	8
3.1	Key Granularity	8
3.2	Eviction and Concurrency	8
3.3	Negative-Result Policy	8
4	Backend: Rate Limiting	8
4.1	Render Reverse-Proxy Awareness	9
5	Backend: AI Triage	9
5.1	Prompt Contract	9
5.2	Rule-Based Fallback	9
5.3	Independence from Search	10
6	Backend: Reliability Guarantees	10
6.1	Catastrophic Failure Path	10
6.2	Middleware Safety Net	10

7	Frontend: Offline Architecture	10
7.1	Tier 1 — Live Backend Search	11
7.2	Tier 2 — localStorage Cache	11
7.3	Tier 3 — Bundled Facilities	11
7.4	Tier 4 — Mock Data	11
7.5	Service Worker	12
7.6	Country Emergency Numbers	12
8	Frontend: The Real-Time Map	12
8.1	Tile Provider	12
8.2	Attribution	12
8.3	User Marker	12
8.4	Service Markers	13
8.5	Re-Centering	13
9	Frontend: Internationalization	13
9.1	Language Coverage	13
9.1.1	Indian Languages (all 22 Schedule-VIII languages plus English)	13
9.1.2	Global Languages (most-spoken per region)	13
9.2	Right-to-Left Support	13
9.3	Selection Logic	14
9.4	Translation Coverage	14
10	Frontend: Crash Detection	14
10.1	Signal 1 — GPS Velocity Collapse	14
10.2	Signal 2 — Accelerometer Spike	14
10.3	Confirmation Logic	14
10.4	Cooldown	14
11	Frontend: SOS Dispatch	14
11.1	WhatsApp-Dominant Countries	15
11.2	SMS-Dominant Countries	15
11.3	Event Telemetry	15
12	Backend API Reference	15
12.1	GET /search	15
12.2	POST /triage	15
12.3	POST /dispatch	15
12.4	GET /health	16
12.5	GET /offline-pack	16
13	Continuous Integration	16
13.1	Frontend CI (frontend-ci.yml)	16
13.2	Backend CI (backend-tests.yml)	16
13.3	PR Guard (pr-guard.yml)	16
13.4	Branch Protection (Applied at the Repository Level)	16
13.5	Test Coverage	17
14	Deployment	17
14.1	Backend on Render	17
14.2	Frontend on Vercel	18
14.3	Bundle Sizes	18
15	Observability and Logging	18
15.1	Request Logging	18
15.2	Service-Level Logs	18
15.3	Suppressed Noise	19
16	Repository Layout	19

17 Key Numbers	21
-----------------------	-----------

2 Backend: The /search Orchestrator

The search endpoint resolves the question “*what emergency services are near these coordinates, and what are their phone numbers?*” It is implemented in `backend/services/search_service.py` as a four-phase asynchronous pipeline.

2.1 Phase 1 — Parallel Geocode and Overpass

Two operations begin simultaneously via `asyncio.gather`: a reverse-geocode against Nominatim and an Overpass query against OpenStreetMap.

```
1 geo_task = asyncio.create_task(_safe_geocode(lat, lon))
2 osm_task = asyncio.create_task(_safe_overpass(lat, lon))
3 geo, osm_contacts = await asyncio.gather(geo_task, osm_task)
```

Listing 1: `search_service.py` — Phase 1 (lines 127–129)

Wall-clock latency is therefore $\max(t_{\text{geocode}}, t_{\text{overpass}})$ rather than the sum. Both are wrapped in `_safe_*` helpers that catch every exception and return a valid fallback shape; failures never propagate.

2.1.1 The Overpass Query Layer

`backend/services/overpass_service.py` builds an Overpass QL query that selects nine categories within a configurable radius (default 5,000 m). The query is structured as a union over tag filters:

```
1 [out:json][timeout:25];
2 (
3   nwr["amenity"="hospital"](around:5000, LAT, LON);
4   nwr["amenity"="clinic"] ["emergency"="yes"](around:5000, LAT, LON);
5   nwr["amenity"="police"](around:5000, LAT, LON);
6   nwr["amenity"="fire_station"](around:5000, LAT, LON);
7   nwr["amenity"="car_repair"](around:5000, LAT, LON);
8   nwr["shop"="car_repair"](around:5000, LAT, LON);
9   nwr["shop"="tyres"](around:5000, LAT, LON);
10  nwr["shop"="car"](around:5000, LAT, LON);
11  nwr["emergency"="ambulance_station"](around:5000, LAT, LON);
12 );
13 out center;
```

Listing 2: Overpass QL — categories targeted (`overpass_service.py`:85–119)

If the first response yields fewer than three results, the radius doubles to 10,000 m and the query repeats. This handles sparse regions without paying for the wider radius on dense urban queries.

2.1.2 Multi-Mirror Retry Strategy

A single Overpass endpoint frequently rate-limits or 502s. The fetcher rotates across three independent mirrors with exponential backoff:

Attempt	Endpoint and Backoff
1	<code>https://overpass-api.de/api/interpreter</code> (no delay)
2	<code>https://overpass.kumi.systems/api/interpreter</code> (2 s)
3	<code>https://overpass.openstreetmap.fr/api/interpreter</code> (4 s)

Each request has a 30-second timeout. If all three mirrors fail, the function returns `[]` rather than raising; downstream phases continue with whatever data exists.

2.1.3 Element Parsing and Distance

Each Overpass element is normalised by `parse_element` (`overpass_service.py`:122–170):

1. Extract `lat`, `lon` (from the element or its `center` field for ways).
2. Compute haversine distance from the user, expressed in kilometres rounded to two decimals.
3. Map the OSM tag to one of nine internal categories: `hospital`, `police`, `ambulance`, `fire`, `towing`, `repair`, `tyre`, `showroom`.
4. Drop results with no name *and* no phone (reduces “Unnamed Building” noise).
5. Parse the `opening_hours` tag into a boolean `isOpen` via `services/hours_parser.py` (supports 24/7, day ranges, time ranges, and holiday tags).

The haversine implementation is unrolled in pure Python (no GIS dependency):

```

1 def haversine(lat1, lon1, lat2, lon2):
2     R = 6371.0 # Earth radius (km)
3     dlat = math.radians(lat2 - lat1)
4     dlon = math.radians(lon2 - lon1)
5     a = (math.sin(dlat / 2) ** 2 +
6         math.cos(math.radians(lat1)) *
7         math.cos(math.radians(lat2)) *
8         math.sin(dlon / 2) ** 2)
9     return 2 * R * math.asin(math.sqrt(a))

```

Listing 3: `overpass_service.py`:66–78

2.1.4 Proximity Deduplication

OSM frequently lists the same facility as both a node (the entrance) and a way (the building polygon). `_dedupe_smart` (`overpass_service.py`:185–210) keeps results that are either more than 50 m apart *or* have substantially different names; otherwise the earlier-seen entry wins.

2.2 Phase 2 — Conditional Google Places Fallback

Phase 2 fires only if Phase 1 produced fewer than three contacts with a dialable phone number. This conserves quota: in dense OSM regions (most of Europe, India, and East Asia), Google is never called.

```

1 phoned_osm = [c for c in osm_contacts if c.get("phone")]
2 google_contacts: list[dict] = []
3 if len(phoned_osm) < 3:
4     google_contacts = await _safe_google(lat, lon, geo.get("country_code"))

```

Listing 4: `search_service.py` — Phase 2 trigger (lines 135–138)

`services/googleplaces_service.py` issues four parallel Nearby Search calls (`hospital`, `police`, `car_repair`, `fire_station`) each capped at five results. Per-result Place Details lookups add the `formatted_phone_number` field. The reverse-geocoded country code from Phase 1 is passed as a region hint to improve relevance.

2.2.1 Multi-Key Rotation

The `GOOGLE_PLACES_API_KEY` environment variable is parsed as a comma-separated list of API keys. Round-robin rotation distributes load across multiple billing accounts:

```

1 def _load_keys() -> list[str]:
2     multi = os.getenv("GOOGLE_PLACES_API_KEY", "")
3     keys = [k.strip() for k in multi.split(",") if k.strip()]
4     if not keys:
5         single = os.getenv("GOOGLE_PLACES_API_KEY", "")
6         keys = [single] if single else []
7     return keys

```

Listing 5: googleplaces_service.py:30–36

If no key is configured, Google is silently disabled — the search still returns whatever Overpass produced.

2.3 Phase 3 — Merge and Deduplicate

The two contact lists are merged and deduplicated by phone digits first, then by lowercased name:

```

1 def deduplicate(contacts: list[dict]) -> list[dict]:
2     out, seen_names = [], set()
3     for c in contacts:
4         if not isinstance(c, dict): continue
5         name_key = (c.get("name") or "").lower().strip()
6         if not name_key or name_key in seen_names: continue
7         if any(phones_match(c.get("phone"), e.get("phone")) for e in out):
8             continue
9         seen_names.add(name_key)
10        out.append(c)
11    return out

```

Listing 6: search_service.py:42–64 — deduplication

`phones_match` (`phone_utils.py`) compares the last ten digits after `phonenumbers-normalisation`. This handles cases where OSM and Google disagree on transliteration (“GS Custom” vs. “gs sustom”) but the phone number is canonical. The merged list is then sorted by distance.

2.4 Phase 4 — Phone Enrichment

For the top six closest results without a phone number, the system performs a Find Place from Text lookup against Google Places, biased to a 500-metre radius. On miss, a fallback Nearby Search within 100 m provides a second chance. Each enrichment costs at most one Place Details billable call, capped at six per search to bound worst-case spend.

2.5 Response Shape

```

1 {
2   "contacts": [
3     {
4       "id": "node/8127364521",
5       "name": "Apollo Hospitals, Bannerghatta",
6       "category": "hospital",
7       "phone": "080-26793000",
8       "lat": 12.8932, "lon": 77.5972,
9       "distance": 1.42,
10      "source": "OpenStreetMap",
11      "isOpen": true
12    }
13  ],
14  "source": "OpenStreetMap + Google Places",
15  "landmark": "Bannerghatta Road, BTM Layout, Bengaluru",

```

```

16  "country_code": "IN",
17  "count": 14
18  }

```

Listing 7: Sample /search response

3 Backend: Caching

`backend/services/cache.py` implements an async-safe in-memory TTL cache with LRU eviction. Three module singletons are configured for the dominant lookups:

Cache	TTL	Max entries	Key strategy
overpass_cache	1 hour	200	<code>round(lat,4)_round(lon,4)_r5000</code>
google_cache	1 hour	200	coordinate key + region code
geocode_cache	24 hours	500	coordinate key only

3.1 Key Granularity

Coordinates are rounded to four decimal places before keying. At the equator one degree of latitude is approximately 111 km, so 10^{-4} degrees $\approx$ 11 metres. Two users standing 10 m apart resolve to the same cache key and share results — semantically correct for “what is near me” queries, and trivially exploitable for warm-cache benchmarks (a repeated query from the same demo location returns in tens of milliseconds).

3.2 Eviction and Concurrency

- **LRU on capacity.** When `len(cache) >= max_entries`, the entry with the oldest timestamp is removed.
- **Opportunistic expiry.** Each `set` call scans for and removes expired entries before inserting.
- **Async lock.** Each cache instance holds an `asyncio.Lock`; concurrent `get/set` operations are serialised per cache.

3.3 Negative-Result Policy

Overpass results with zero phone numbers are not cached. This is deliberate: a phoneless cache hit would short-circuit Phase 2 (Google enrichment), permanently denying the user phone-equipped contacts they could otherwise get. The cost is recomputation for sparse regions, accepted as a quality-over-latency trade.

4 Backend: Rate Limiting

`services/rate_limiter.py` implements a per-IP token bucket. Each client receives a bucket of capacity 10 that refills at the configured rate; requests consume one token. When the bucket is empty, the request receives 429 Too Many Requests with a `Retry-After` header in seconds.

Endpoint	Rate	Burst	Notes
GET /search	30 req/min	10	Per-IP
POST /triage	20 req/min	10	Per-IP, lower because AI calls are costlier

4.1 Render Reverse-Proxy Awareness

Render terminates TLS at its edge and forwards requests with an `X-Forwarded-For` header. The default ASGI `request.client.host` reports the Render edge IP, which would collapse all users into a single bucket. `get_client_ip` (`rate_limiter.py:70`) prefers the first entry of `X-Forwarded-For` when present:

```
xff = request.headers.get("x-forwarded-for", "")
return xff.split(",")[0].strip() if xff else request.client.host
```

Buckets are stored in a plain Python dictionary protected by `asyncio.Lock`. They reset whenever the Render dyno cold-starts (free tier idle timeout: 15 minutes).

5 Backend: AI Triage

`backend/services/ai_triage.py` reorders a contact list to match the user's situation. The model is `gemini-2.5-flash`, called via direct REST against <https://generativelanguage.googleapis.com/v1beta/models/> (no SDK dependency — uses the same `httpx` client as the rest of the backend). The free tier allows 15 requests per minute and 1500 requests per day with no billing requirement. Two boolean situational flags are accepted: `injured` and `blocking_traffic`. JSON mode is opt-in via `generationConfig.responseMimeType = "application/json"`.

5.1 Prompt Contract

The system prompt mandates a strict JSON response with exactly two top-level keys:

```
1 {
2   "contacts": [<same items as input, reordered>],
3   "reason": "<one short sentence explaining the top pick>"
4 }
```

Listing 8: Required model output schema

Validation checks (`ai_triage.py:87–99`) that:

1. the response parses as JSON,
2. `contacts` is a list of exactly the same length as the input,
3. every contact carries the same `id` as an input entry,
4. `reason` is a non-empty string.

5.2 Rule-Based Fallback

If the model call fails (no API key, network error, malformed response, validation failure, count mismatch), a deterministic priority table takes over:

Situation	Priority order (descending)
Injured & blocking	ambulance, hospital, police, towing, repair
Injured only	ambulance, hospital, police, repair, towing
Blocking only	police, towing, ambulance, hospital, repair
Neither	repair, tyre, police, towing, hospital, ambulance

The fallback is the same logic the model is prompted to follow, so the user-visible behaviour is consistent whether or not the model responded.

5.3 Independence from Search

Triage is a separate endpoint (POST /`triage`) called only after the user answers the situational questions. The map and contact list render with no AI dependency; if Gemini is entirely unreachable, the list is still useful.

6 Backend: Reliability Guarantees

Every external call is wrapped in a `_safe_*` helper that catches all exceptions and returns a valid fallback shape:

Wrapper	Fallback return
<code>_safe_overpass</code>	<code>[]</code>
<code>_safe_google</code>	<code>[]</code>
<code>_safe_geocode</code>	<code>{"landmark": "latř, lonř", "country_code": null}</code>
<code>deduplicate</code>	Skips non-dict items rather than raising
<code>triage_service.prioritise</code>	Rule-based ordering with outer try/except as last resort

6.1 Catastrophic Failure Path

If every upstream is unavailable simultaneously:

```
{
  "contacts": [],
  "source": "none (upstreams unavailable)",
  "landmark": "13.0827, 80.2707",
  "country_code": null,
  "count": 0
}
```

Listing 9: Response when all upstreams fail

The HTTP status is still 200 OK. The frontend interprets an empty result as a trigger for the offline fallback chain (Section 7).

6.2 Middleware Safety Net

`backend/middleware.py` adds three layers around every request:

1. **RequestIDMiddleware.** Stamps every request with a UUID-v4 (or echoes the client's X-Request-ID) and exposes it on the response.
2. **RequestLogMiddleware.** Emits one log line per request: `method path status duration_ms request_id[:8]`.
3. **ErrorHandlingMiddleware.** Outermost. Catches anything an inner handler raised, logs the full traceback at ERROR with the request ID, and returns 503 Service Unavailable with the request ID in the JSON body for support reporting.

7 Frontend: Offline Architecture

The frontend implements a four-tier fallback chain. Each tier is queried only if the preceding tier has no answer. The order is fixed in `frontend/src/App.jsx` (lines 197–255).

7.1 Tier 1 — Live Backend Search

`searchNearby(lat, lon)` in `src/utils/overpass.js` posts to `/search?lat=&lon=` on the configured backend. Timeout: 8 seconds. On success the response is also written to Tier 2 for future offline use.

7.2 Tier 2 — localStorage Cache

`src/utils/offlineDB.js` stores responses in `window.localStorage` keyed by a coarsened coordinate grid:

```
1 const TTL_MS = 24 * 60 * 60 * 1000; // 24 hours
2
3 function locationKey(lat, lon) {
4   // Round to 2 decimal places (~1.1 km grid)
5   return `roadsos_cache_${lat.toFixed(2)}_${lon.toFixed(2)}`;
6 }
```

Listing 10: `offlineDB.js`: 24-hour client-side cache

The grid is 1.1 km square at the equator. A user who searched yesterday from any point within the same square gets a cache hit today. Entries older than the TTL are removed on read.

7.3 Tier 3 — Bundled Facilities

When network requests fail and no cache entry exists, `src/utils/bundledFacilities.js` computes the nearest entries from a JSON file shipped in the build bundle: `src/data/bundled_facilities.json` (818 entries, 200 countries).

Region	Records	Notes
India (deepest)	~40	Tamil Nadu + Delhi/Mumbai/Bengaluru/Hyderabad metros
North America	22	US trauma centers (Level I + II), Canadian provinces
Europe	38	EU member trauma units + UK NHS major centres
East and Southeast Asia	26	Japan, China, Korea, Thailand, Vietnam, Indonesia, Philippines, Singapore, Malaysia
Middle East and North Africa	18	UAE, Saudi Arabia, Israel, Turkey, Iran, Egypt
Sub-Saharan Africa	32	At least one per UN-recognised state
Latin America & Caribbean	24	Brazil/Argentina/Mexico majors plus smaller-nation hospitals
Oceania	8	Australia, New Zealand, Pacific island states

Search radius starts at 80 km and expands to 600 km if zero results match. Distance is computed by the same haversine formula used server-side, ensuring frontend and backend agree on ordering.

7.4 Tier 4 — Mock Data

If all three tiers above return empty, a hardcoded array of seven Bengaluru-area contacts (`App.jsx:41–112`) is used as a last visual placeholder. This tier exists so the UI is never blank; in production deployments it would be removed under a build flag.

7.5 Service Worker

`frontend/public/sw.js` is registered through `vite-plugin-pwa` with the `injectManifest` strategy. The Workbox configuration includes:

- **Precache.** The full app shell (HTML, JS, CSS, fonts) is injected via `self.__WB_MANIFEST` at build time. Total: roughly 670 KiB across six entries.
- **Runtime cache for `/search`.** `NetworkFirst` with an 8-second timeout, 50-entry LRU, 24-hour max age. After the timeout, the SW serves the most recent cached response for that URL.
- **Runtime cache for `/offline-pack`.** `CacheFirst` with a single entry and seven-day max age. The backend's offline pack is downloaded once and reused.
- **`skipWaiting()` on install.** Critical for live deployments. Without it, a new build would only take effect after the user closed every open tab.

7.6 Country Emergency Numbers

Separate from facility lookup, `src/utils/emergencyNumbers.js` is a compile-time mapping from ISO-3166 alpha-2 country code to a list of national emergency dial codes. Examples: IN → [108, 100, 101, 112], US → [911], GB → [999, 112], DE → [112, 110]. The banner at the top of the app renders these in the user's language and is fully offline-capable.

8 Frontend: The Real-Time Map

`frontend/src/components/RealMap.jsx` renders an OpenStreetMap-backed Leaflet map centred on the user's GPS coordinates.

8.1 Tile Provider

Tiles are fetched from CartoDB's Dark Matter raster style:

```
https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png
```

CartoDB serves these tiles free of charge for non-commercial use with attribution. No API key is required. The `{s}` placeholder distributes across the subdomains `a-d`. The `{r}` suffix delivers Retina (`@2x`) tiles on high-DPI displays.

8.2 Attribution

OSM's Open Database License (ODbL) and CartoDB's terms both require visible attribution on derived works. The Leaflet attribution control renders bottom-right:

```
&copy; <a href="https://www.openstreetmap.org/copyright">OSM</a>
&copy; <a href="https://carto.com/attributions">CARTO</a>
```

8.3 User Marker

A custom `L.divIcon` draws three concentric layers:

1. **Halo.** 110 px radial gradient at 33% opacity, providing visual mass.
2. **Pulse ring.** 28 px circle animated via `@keyframes rs-blip` (2.5 s ease-out infinite).
3. **Dot.** 18 px solid circle with a 3 px white border and a coloured drop-shadow.

When the GPS signal is older than the freshness threshold, the colour shifts from green (`#22C55E`) to grey (`#94A3B8`) and the pulse animation continues at reduced opacity.

8.4 Service Markers

Up to six contacts are rendered as map pins using their actual lat/lon coordinates from the search response. Each pin combines:

- A coloured circle (red for medical: hospital, ambulance, fire; blue for police; teal for mechanical: towing, repair, tyre, showroom)
- A category emoji
- A floating chip above with the truncated facility name and distance in kilometres

The map is configured non-interactively in the hero section: `dragging`, `scrollWheelZoom`, `doubleClickZoom`, `touchZoom`, `boxZoom`, and `keyboard` are all disabled. This preserves the SOS button as the primary interaction. A separate full-screen mode (not yet shipped) would enable interaction.

8.5 Re-Centering

A child component, `MapRecenter`, hooks into `useMap()` and calls `map.setView()` with a 0.6-second animation whenever the location prop changes. This makes the demo location picker (Bengaluru → London → Tokyo) glide smoothly across the globe.

9 Frontend: Internationalization

`frontend/src/i18n/` contains 43 translation bundles registered through `i18next`.

9.1 Language Coverage

9.1.1 Indian Languages (all 22 Schedule-VIII languages plus English)

Assamese, Bengali, Bodo, Dogri, English, Gujarati, Hindi, Kannada, Kashmiri, Konkani, Maithili, Malayalam, Manipuri, Marathi, Nepali, Odia, Punjabi, Sanskrit, Santali, Sindhi, Tamil, Telugu, Urdu.

9.1.2 Global Languages (most-spoken per region)

Spanish, French, Portuguese, German, Italian, Dutch, Polish, Russian, Ukrainian, Romanian, Greek, Turkish, Arabic, Persian, Hebrew, Swahili, Amharic, Indonesian, Malay, Thai, Vietnamese, Chinese, Japanese, Korean, Filipino.

9.2 Right-to-Left Support

Six languages set the document direction to RTL on selection (Arabic, Hebrew, Persian, Urdu, Kashmiri, Sindhi). The handler in `src/i18n/index.js`:

```
1 function applyDir(code) {  
2   const loc = getLocale(code);  
3   document.documentElement.lang = loc.bcp47 || code;  
4   document.documentElement.dir = loc.dir || 'ltr';  
5 }
```

Listing 11: `i18n/index.js` — `applyDir`

9.3 Selection Logic

On first launch, the user sees a full-screen modal listing all 48 languages, grouped into “India” (22) and “World” (26) sections. There is no automatic pre-selection based on GPS country code; the user picks explicitly. The choice is persisted to `localStorage['roadsos:lang']` and never re-asked.

If the user dismisses the picker without choosing, the language defaults to:

1. The previous session’s choice from `localStorage`, if any.
2. `navigator.language` (first two characters), if a translation bundle exists for it.
3. `en`.

9.4 Translation Coverage

Each bundle contains approximately 40 translation keys spanning SOS button states, emergency-category labels, location-status messages, action-button labels, crash-alert prompts, and Medical ID form fields. The complete key list is enumerated in [src/i18n/en.json](#) as the canonical reference.

10 Frontend: Crash Detection

[frontend/src/hooks/useLocation.js](#) fuses two sensor signals to identify a likely crash without requiring dedicated hardware.

10.1 Signal 1 — GPS Velocity Collapse

The hook maintains a rolling buffer of GPS samples (each containing speed in m/s and a timestamp). A velocity collapse is registered when sustained samples at ≥ 40 km/h are followed within 2.5 seconds by a sample at ≤ 5 km/h (tuned to reject ordinary braking and stop-and-go traffic).

10.2 Signal 2 — Accelerometer Spike

A `devicemotion` listener tracks `accelerationIncludingGravity`. A spike is registered when the magnitude exceeds 3.5 G (34.3 m/s²). On iOS 13 and later, motion permission must be granted by an explicit user gesture; the first tap on the SOS area triggers `requestMotionPermission()`.

10.3 Confirmation Logic

A crash is reported only if both signals occur within 4 seconds of each other. The two-signal requirement substantially reduces false positives from phone drops (large G spike, no velocity change) and potholes (small velocity change, no sustained G spike).

10.4 Cooldown

After firing, the detector enters a 12-second cooldown during which neither signal can re-trigger the alert. This prevents an aftershock or a follow-up impact from doubling notifications.

11 Frontend: SOS Dispatch

[frontend/src/utils/sosDispatch.js](#) translates a button tap into a native messaging app deep link, choosing the dominant medium per country.

11.1 WhatsApp-Dominant Countries

For countries where WhatsApp is the de-facto messaging app, the handler issues `window.open('https://wa.me/<phone number>?text=<encoded message>')`. The deep link opens WhatsApp (or its web client) pre-filled with:

- The phrase “Emergency. I need help.”
- GPS coordinates as decimal degrees.
- A Google Maps link with the same coordinates as the query string.
- The reverse-geocoded landmark string, if available.

WhatsApp-dominant set includes IN, BR, GB, NG, ZA, MX, ID, ES, IT, AR, CO, CL, PE, EG, SA, AE, MY, TR, and roughly 70 others. Because `wa.me` accepts only one recipient per URL, this path sends to a single contact at a time.

11.2 SMS-Dominant Countries

For countries dominated by SMS (US, CA, FR, DE, JP, KR, AU, and most of Northern Europe), the handler issues `window.location.href = 'sms:<contact1>,<contact2>,...?body=<encoded message>'`. The native SMS composer opens with all configured emergency contacts pre-populated as recipients.

11.3 Event Telemetry

After dispatch, the page emits a `roadsos:sos-sent` CustomEvent with details about the recipient and the medium chosen. `App.jsx` listens for this event and opens `DispatchScreen.jsx` for the post-send confirmation UI. The same event triggers a fire-and-forget POST `/dispatch` to the backend for telemetry.

12 Backend API Reference

12.1 GET /search

Parameter	Description
<code>lat</code>	Float, WGS-84 latitude, range $[-90, 90]$. Required.
<code>lon</code>	Float, WGS-84 longitude, range $[-180, 180]$. Required.
Returns a JSON object with the shape given in Section 2.5. Rate limit: 30 req/min/IP.	

12.2 POST /triage

```
{
  "contacts": [<list of contact objects from /search>],
  "injured": true,
  "blocking_traffic": false
}
```

Listing 12: Request body

Returns the same contact list, reordered, plus a `reason` string of one short sentence. Rate limit: 20 req/min/IP.

12.3 POST /dispatch

Telemetry sink. Logs the dispatch event server-side with request ID and returns 202 Accepted. No body validation beyond JSON parsing.

12.4 GET /health

Returns `{"status": "ok", "version": "1.x.x", "timestamp": "2026-05-16T..."}`. Used by uptime monitors and by the frontend's `startBackendWarmup()` to wake the Render dyno on app load.

12.5 GET /offline-pack

Returns the bundled facility JSON for client-side warm caching. `Cache-Control: public, max-age=604800` (7 days).

13 Continuous Integration

Three GitHub Actions workflows are defined in `.github/workflows/`.

13.1 Frontend CI (`frontend-ci.yml`)

- **Triggers.** push and pull_request against main when `frontend/**` is touched.
- **Matrix.** Node.js 20 and 22 in parallel.
- **Steps.** `npm ci --legacy-peer-deps → vite build → vitest run`.
- **Artifact.** The Node-20 job uploads `frontend/dist` as a build artifact (7-day retention).

13.2 Backend CI (`backend-tests.yml`)

Three jobs run sequentially:

1. **lint.** `ruff check` (rule set: E, W, F, I, UP, B, C4, SIM) followed by `ruff format --check`. Four rules are explicitly ignored: E501 (line length), E402 (intentional `load_dotenv()` ordering), B008 (FastAPI `Depends()`), and SIM108 (ternary readability).
2. **startup-check.** Single-step verification that `python -c "import main"` succeeds against `requirements.txt` with dummy API keys — catches dependency-chain breakage before tests run.
3. **test.** `pytest` on Python 3.11 and 3.12 in parallel. Depends on lint and startup-check passing.

13.3 PR Guard (`pr-guard.yml`)

Two jobs run on every pull request against main:

1. **conflict-check.** Attempts a no-commit, no-fast-forward merge of the PR branch against `origin/main`. Fails if Git reports `CONFLICT`, surfacing the issue before review.
2. **branch-staleness.** Counts commits the PR branch is behind main via `git rev-list --count HEAD..origin/main`. If the count exceeds 30, emits a warning recommending a rebase. This prevents the kind of large-scale merge conflict that arose during the merge of `arindam-branch-` into main.

13.4 Branch Protection (Applied at the Repository Level)

The main branch enforces:

- Required status checks: Build & Test (Node 20), Build & Test (Node 22), pytest (Python 3.11), pytest (Python 3.12), Lint (ruff), App startup check, and Merge conflict check.
- Strict mode: branches must be up to date with main before merging.

- No force pushes.
- No branch deletions.

13.5 Test Coverage

Backend tests are organised by service:

Test file	LOC	Scope
test_search_service.py	63	deduplication logic: invalid dicts, empty names, case-insensitive name match, phone equality
test_ai_triage.py	90	rule-based fallback, JSON schema validation, single- and two-leg priority orderings
test_overpass.py	207	element parsing, haversine accuracy, proximity dedup, category classification
test_overpass_parsing.py	—	edge-case element shapes (nwr without center, tags missing)
test_rate_limiter.py	—	token-bucket refill, X-Forwarded-For extraction, retry-after header
test_cache.py	—	TTL expiry, LRU eviction, hit/miss statistics
test_geocoder.py	—	ISO country-code validation against the regex
test_hours_parser.py	—	opening_hours parsing: 24/7, day ranges, time ranges, public holidays
test_phone_utils.py	—	E.164 normalisation, is_dialable (3–15 digits), phones_match
test_middleware.py	—	request-ID stamping, log line format, error handler 503 response

14 Deployment

14.1 Backend on Render

`render.yaml` at the repository root defines a single web service:

```

1 services:
2   - type: web
3     name: roadsos-api
4     runtime: python
5     rootDir: backend
6     plan: free
7     buildCommand: pip install -r requirements.txt
8     startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
9     envVars:
10      - key: GEMINI_API_KEY          # required, free at aistudio.google.com
11        sync: false
12      - key: GOOGLE_PLACES_API_KEY  # optional, comma-separated for
13        multi-key
14        sync: false
15      - key: PYTHON_VERSION
16        value: 3.11.9

```

Listing 13: render.yaml

Free-tier specifics:

- Dyno sleeps after 15 minutes idle; first request triggers a ~30-second cold start.
- No persistent disk (acceptable; all data is upstream or in client browser).

- TLS terminated at edge; backend sees X-Forwarded-For only.

`startBackendWarmup()` in the frontend fires a GET `/health` immediately on app mount, paying the cold-start cost before the user runs their first search.

14.2 Frontend on Vercel

`vercel.json`:

```

1 {
2   "version": 2,
3   "buildCommand": "cd frontend && npm install && npm run build",
4   "outputDirectory": "frontend/dist",
5   "installCommand": "echo 'install handled in buildCommand'",
6   "framework": null,
7   "rewrites": [
8     { "source": "/(.*)", "destination": "/index.html" }
9   ]
10 }
```

Listing 14: `vercel.json`

The single rewrite ensures client-side routes resolve through `index.html` (SPA behaviour). Build environment variables:

- `VITE_API_URL` — e.g. `https://roadsos-api.onrender.com`

14.3 Bundle Sizes

Asset	Raw	Gzip
<code>dist/assets/index.js</code>	610 KiB	181 KiB
<code>dist/assets/index.css</code>	80 KiB	18 KiB
<code>dist/sw.js</code>	25 KiB	8 KiB
<code>dist/index.html</code>	0.8 KiB	0.4 KiB
Precache total	—	~210 KiB

15 Observability and Logging

15.1 Request Logging

Every request emits one structured log line:

```
2026-05-16T14:32:08Z [INFO] roadsos.request GET /search 200 1247ms a8f3e9c2
```

Format: timestamp, level, logger, method, path, status, duration in milliseconds, first eight characters of the request ID.

15.2 Service-Level Logs

Service	Notable log events
<code>overpass_service</code>	retry attempts (WARN), cache hits (INFO), per-fetch summary (INFO)
<code>googleplaces_service</code>	enrichment count per <code>/search</code> (INFO), key rotation (DEBUG)
<code>geocoder</code>	invalid country code from Nominatim (WARN)
<code>ai_triage</code>	API call failure with exception type (WARN), JSON validation failure (WARN)
<code>rate_limiter</code>	per-IP throttling events (INFO)

15.3 Suppressed Noise

`logging_config.py` silences `httpx`, `httpcore`, and `anyio` below `WARN` level. These otherwise emit a connection-lifecycle line for every internal HTTP call.

16 Repository Layout

```
Roadproj/
|- backend/
|   |- main.py                # FastAPI app entry, middleware wiring
|   |- middleware.py          # RequestID, RequestLog, ErrorHandling
|   |- logging_config.py      # log format + library noise suppression
|   |- requirements.txt        # runtime deps (FastAPI, httpx, phonenumbers
|   |   , ...)
|   |- requirements-dev.txt    # adds pytest, pytest-asyncio, ruff
|   |- pytest.ini              # asyncio_mode = auto, testpaths
|   |- ruff.toml               # lint rule set + project ignores
|   |- services/
|       |- search_service.py  # /search orchestrator (4 phases)
|       |- overpass_service.py # OSM Overpass client + parser
|       |- googleplaces_service.py # Google Places + multi-key rotation
|       |- geocoder.py        # Nominatim reverse geocode
|       |- ai_triage.py        # Gemini 2.0 Flash + rule fallback
|       |- triage_service.py   # /triage router
|       |- cache.py            # async TTL/LRU cache
|       |- rate_limiter.py     # per-IP token bucket
|       |- phone_utils.py      # phonenumbers wrapper
|       |- hours_parser.py     # OSM opening_hours parser
|       |- health_service.py   # /health endpoint
|       |- dispatch_service.py # /dispatch telemetry endpoint
|       |- 'offline_service.py # /offline-pack endpoint
|   |- 'tests/                 # 10 test files, pytest discoverable
|- frontend/
|   |- vite.config.js          # Vite + vite-plugin-pwa configuration
|   |- vitest.config.js        # test runner config
|   |- public/sw.js            # Workbox service worker source
|   |- 'src/
|       |- App.jsx             # top-level orchestrator + state machine
|       |- main.tsx            # ReactDOM root, i18n init
|       |- final-design.css     # design system: colours, typography, layout
|       |- components/
|           |- MapHero.jsx      # hero section (map + dock + SOS)
|           |- RealMap.jsx      # Leaflet map + custom markers
|           |- SOSButton.jsx    # SOS dispatch with WhatsApp/SMS routing
|           |- LanguagePicker.jsx # first-launch 43-language modal
|           |- MedicalIdModal.jsx # blood type, allergies, contacts
|           |- CrashAlert.jsx   # post-crash confirmation UI
|           |- DispatchScreen.jsx # post-SOS confirmation UI
|           |- TriageModal.jsx  # injured + blocking questions
|           |- CountryEmergency.jsx # 108/112/911 banner
|       |- '...
|       |- hooks/
|           |- useLocation.js    # GPS + motion + crash detection
|           |- 'useNetwork.js    # online/offline status
|       |- utils/
|           |- overpass.js       # /search client
|           |- googlePlaces.js   # /triage client
|           |- offlineDB.js      # localStorage cache
|           |- bundledFacilities.js # tier-3 fallback search
|           |- emergencyNumbers.js # country -> dial codes
```

```

|   |   |- sosDispatch.js      # WhatsApp/SMS deeplink builder
|   |   |- medicalId.js       # on-device emergency-contact storage
|   |   |- plusCodes.js       # OLC encoder (pure JS, no deps)
|   |   '- backendWarmup.js    # Render cold-start mitigation
|   |- i18n/
|   |   |- index.js           # i18next init + RTL handler
|   |   |- locales.js         # locale metadata (native, English, dir)
|   |   |- countryLanguage.js # ISO country -> language code
|   |   '- *.json             # 43 translation bundles
|   '- data/
|       '- bundled_facilities.json # 818 entries, 200 countries
|- docs/
|   |- ARCHITECTURE.md        # architecture review + ADRs
|   '- TECHNICAL.tex          # this document
|- .github/workflows/
|   |- frontend-ci.yml        # build + test on Node 20, 22
|   |- backend-tests.yml      # lint + startup + pytest
|   '- pr-guard.yml           # merge conflict + staleness detection
|- render.yaml                # Render service definition
|- vercel.json                 # Vercel build configuration
|- LICENSE                     # MIT
'- README.md                   # human-facing overview

```

17 Key Numbers

Metric	Value
Countries with at least one bundled facility	200
Bundled facilities total	818
Languages shipped	43
Indian Schedule-VIII languages covered	22 of 22
RTL languages supported	6 (Arabic, Persian, Hebrew, Urdu, Kashmiri, Sindhi)
OSM categories queried in parallel	9
Overpass mirrors with retry	3
Overpass timeout per attempt	30 s
Default Overpass radius	5,000 m (expands to 10,000 m if sparse)
Bundled-fallback radius	80 km (expands to 600 km if sparse)
Cache TTL — Overpass / Google / geocode	1 h / 1 h / 24 h
Cache key granularity	~11 m (4 decimal places)
localStorage cache TTL	24 h
localStorage cache grid	~1.1 km
Rate limit — /search	30 req/min/IP
Rate limit — /triage	20 req/min/IP
Phone enrichment cap	6 Place Details calls per /search
Crash detection — velocity collapse threshold	sustained ≥ 40 km/h $\rightarrow$ ≤ 5 km/h within 2.5 s
Crash detection — accelerometer threshold	≥ 3.5 G
Crash detection — signal alignment window	4 s
Crash detection — post-alert cooldown	12 s
AI model	gemini-2.5-flash (free tier: 15 RPM / 1500 RPD)
Frontend bundle (gzip)	181 KiB JS + 18 KiB CSS
Service worker precache entries	6 (~210 KiB)

Document version. Generated from `main` at commit `f94dc2a`. Re-run after substantive code changes; this is a hand-maintained reference, not auto-generated.